Three Axes of Quantum Risk: A Unified Observability Model for PQC, QKD, and Crypto-Agility

Aleaddin Özer (CIO, E2E Solutions)
Murat Aydos (Assoc. Prof., Hacettepe University)

2026-05-13

ABSTRACT

Abstract

The cryptographic protections that secure today's networks are scheduled for forced retirement. NIST's Module-Lattice and Hash-Based signature/KEM standards (FIPS 203, 204, 205) ship as production algorithms; the U.S. NSA's CNSA 2.0 timetable mandates broad migration by 2030–2035; and a parallel hardware track — Quantum Key Distribution governed by ETSI GS QKD 014 — offers an orthogonal control on high-assurance links. Operators preparing for this transition face three independent questions that current tooling answers separately, if at all: (i) which cryptographic primitives in my environment are algorithmically quantum-resistant, (ii) which channels are additionally protected by quantum-secure key delivery, and (iii) how agile is each asset — can it be migrated by configuration, library upgrade, or only by firmware replacement?

We argue that quantum-risk posture is a three-axis problem, not a one-axis problem, and that observability tooling should reflect this. We formalize a posture model that grades each cryptographic asset on algorithmic resistance (A), channel protection (C), and migration agility (G), and combines them with the operator's deadline horizon into a single deadline-adjusted quantum-risk score. We extend an open event schema (Sezar `crypto_inventory_event v1`) with the additional axes, present a reference architecture spanning eBPF-based TLS observation, an ETSI GS QKD 014 collector with a reusable Key-Management-Entity emulator, and a static crypto-agility scanner. We characterize the system through three empirical studies that any practitioner can replicate: (1) a TLS handshake survey of the Tranco-top-1k against the Internet's PQ-ready hosts; (2) a controlled ETSI 014 emulator study covering link, KME, and application-layer failure modes; (3) a crypto-agility audit of fifty widely deployed open-source server projects under a published scoring rubric.

Our contributions are: (i) the three-axis quantum-risk posture model and its deadline-adjusted scoring function; (ii) an open event schema and reference implementation; (iii) an open ETSI GS QKD 014 emulator and corpus of replay scenarios; (iv) a published, reproducible crypto-agility scoring rubric and ruleset; (v) the first empirical baseline that grades real-world systems on all three axes simultaneously.

1. Introduction

The cryptography deployed on the open Internet today was designed under assumptions that will not survive the next decade of computing. Shor's algorithm on a sufficiently large fault-tolerant quantum computer breaks RSA, finite-field Diffie–Hellman, and every deployed-at-scale elliptic curve scheme in polynomial time

[1]. Grover's algorithm halves the effective key length of symmetric primitives [2]. The exact arrival date of a *cryptographically relevant quantum computer* (CRQC) remains contested, but two operational facts are no longer in doubt. First, adversaries can record encrypted traffic now and decrypt it later once a CRQC is available — the *harvest-now, decrypt-later* threat [3], [4]. Second, the standards bodies have moved: NIST finalized FIPS 203 (ML-KEM, née Kyber), FIPS 204 (ML-DSA, née Dilithium), and FIPS 205 (SLH-DSA, née SPHINCS+) in 2024 [5], [6], [7]. The U.S. NSA's CNSA 2.0 suite gives calendar deadlines: software/firmware signing in PQC by 2025, browsers and servers by 2030, network equipment by 2030, with operating systems and other classes by 2033 [3]. The U.K., EU, and ANSSI guidance follows similar curves [8], [9].

This places operators in an unfamiliar position. They must inventory, classify, and migrate cryptographic assets across networks they did not entirely design, on a deadline they did not set, using algorithms that did not exist in deployable form three years ago. Tools for the first step — inventory and classification — are appearing [10], [11], [12], [13], and standards bodies have begun publishing guidance on which observables matter [4], [14].

In parallel, a hardware-rooted track has matured. Quantum Key Distribution (QKD), proposed in 1984 [15] and matured through the SECOQC, Tokyo, and EuroQCI testbeds [16], [17], [18], delivers symmetric key material through a channel whose security reduces to physics rather than computational assumptions. ETSI's *GS QKD 014* specification standardized the REST interface between QKD key-management entities (KMEs) and the applications that consume their keys [19]. Commercial deployments exist in finance, government, and metropolitan-fiber settings [20], [21], [22].

Industrial and academic literature treats these two tracks — PQC and QKD — as alternatives. Either you migrate your algorithms (PQC), or you install QKD links on high-assurance segments, with the implicit understanding that QKD's hardware cost and distance limits make it inappropriate for general use [23], [24]. We argue that this framing collapses too soon. From an *observability* standpoint — that is, from the standpoint of an operator trying to know what risks are present in their environment on any given day — PQC and QKD are *complementary observables*, not alternatives. An asset using ECDSA-P256 over a fiber link protected by a Toshiba MU-QKD3 system is in a materially different posture than an asset using ECDSA-P256 over an unprotected wide-area link, even though their algorithms are identical. A unified observability model must capture both.

We argue further that a complete quantum-risk model must include a third axis that current tooling almost entirely ignores: *crypto- agility*. An asset using a classical algorithm today but whose algorithm is selected at runtime by a server configuration field is in a very different posture than an asset whose algorithm is hard-coded into firmware that ships from a vendor and rotates only on hardware refresh. The first is a configuration change; the second is a hardware-replacement program. The NIST PQ-migration playbook already names crypto-agility as a precondition for orderly migration [4], [25], yet no production observability tool we have seen surfaces it as a first-class telemetry axis. Without it, operators answering "are we PQ-ready?" are answering a question one order of magnitude easier than the one their deadline requires: "can we *become* PQ-ready before the deadline?"

This paper argues for, and provides a reference implementation of, a **three-axis quantum-risk observability model**:

- **Axis A — Algorithmic Resistance.** Is the primitive itself quantum-resistant under standard assumptions?
- **Axis C — Channel Protection.** Is the key material protected by a quantum-secure delivery mechanism (QKD, hybrid PSK derived from QKD)?
- **Axis G — Migration Agility.** How quickly can axis A be changed on this asset — by configuration, library upgrade, or hardware replacement?

We collapse the three into a single deadline-adjusted quantum-risk score $q(asset, t)$ suitable for dashboard display, alert thresholds, and inter-organization comparison. We extend the open Sezar `crypto_inventory_event v1` schema with the additional axes, present a reference implementation built around five cooperating agents emitting one event shape, and we evaluate the system through three empirical studies designed to be reproducible by any practitioner with a Linux host, a public Internet connection, and the patience to scan one thousand hosts.

1.1 Contributions

This paper makes the following contributions:

1. **A three-axis quantum-risk posture model.** We argue that algorithmic resistance, channel protection, and migration agility are three independent observables that together determine an asset's actual exposure to the post-quantum transition. We formalize each axis on a defined scale and define the unified deadline-adjusted risk function (§5).
2. **An extended open event schema.** We extend the Sezar `crypto_inventory_event v1` schema with channel-protection and agility fields, and we add new asset kinds for QKD link and KME observations. The extensions are additive and non-breaking (§6).
3. **A reference architecture and implementation.** We present Sezar, an open-source reference observability platform spanning five agents (network, certificate, blockchain, key-management, QKD), a posture rollup library, and a collector/dashboard. We detail the implementation choices that allow a single Rust workspace to host wire-level eBPF observation, REST-based QKD telemetry collection, and static crypto-agility analysis under one event shape (§7).
4. **An ETSI GS QKD 014 emulator and replay corpus.** We provide an open implementation of the ETSI GS QKD 014 v1.1.1 Key Delivery API with synthetic key generation, configurable QBER, link state-change replay scenarios, and a documented capture format. The emulator enables practitioners and researchers without access to physical QKD hardware to drive and test QKD-aware software (§7.2, §8.2).
5. **A reproducible crypto-agility scoring rubric.** We define a five-level ordinal scale (Negotiated / Configurable / Pinned / Locked / Frozen), publish a Semgrep rule pack that derives a defensible score from source code or installed-package inspection, and release a hand-graded ground-truth corpus of fifty widely deployed open-source projects (§7.3, §8.3).
6. **First empirical baseline grading real-world systems across all three axes.** We report A, C, G scores for the Tranco-top-1k over TLS, for a controlled multi-KME ETSI 014 testbed exercising representative failure modes, and for fifty popular open-source server projects (§8).

1.2 Non-goals

This paper does not propose a new PQC primitive, a new QKD protocol, or a new key-management protocol. It does not argue that QKD should replace PQC, or vice versa; we take the standards-body position that hybrid deployments are the realistic operating mode and instrument both observables. We do not claim to inventory all deployed cryptography on Earth; our empirical work is bounded by the corpora we publish. We do not address adversary modeling beyond accepting the harvest-now/decrypt-later assumption.

1.3 Roadmap

§2 reviews the standards landscape (PQC, QKD, crypto-agility) at the level required to make the paper self-contained. §3 surveys related observability work and identifies the gap. §4 states the threat model and operator assumptions. §5 defines the three-axis posture model. §6 specifies the event schema. §7 presents the Sezar reference implementation. §8 reports the empirical evaluation. §9 discusses limitations, ethics, and deployment guidance. §10 concludes.

2. Background

2.1 Post-Quantum Cryptography Standardization

NIST initiated its post-quantum cryptography standardization process in 2016 [26] and finalized the first three production standards in August 2024 [5], [6], [7]. These are:

- **FIPS 203 — ML-KEM** (formerly CRYSTALS-Kyber), a module-lattice key-encapsulation mechanism at three parameter sets corresponding to NIST security categories 1, 3, and 5.
- **FIPS 204 — ML-DSA** (formerly CRYSTALS-Dilithium), a module-lattice digital signature scheme also at three parameter sets.
- **FIPS 205 — SLH-DSA** (formerly SPHINCS+), a stateless hash-based digital signature scheme, with both small and fast parameter sets across SHA2 and SHAKE hashes.

A fourth standard for a fast falcon-style lattice signature (FN-DSA, formerly Falcon) is in advanced draft [27]. NIST has additionally signaled that further KEM candidates will be standardized to diversify mathematical assumptions [28]. The U.S. NSA's *Commercial National Security Algorithm Suite 2.0* (CNSA 2.0) identifies ML-KEM-1024, ML-DSA-87, SLH-DSA-SHA2-256s, AES-256-GCM, and SHA-384 / SHA-512 as the required suite for national-security systems by milestones falling between 2025 and 2035 depending on equipment class [3]. The German BSI, French ANSSI, and U.K. NCSC have published equivalent guidance with broadly similar timelines and a shared recommendation to deploy *hybrid* (classical + PQC) key exchange during the transition window [8], [9], [14].

The deployment story is mixed. Chrome and Firefox have shipped X25519MLKEM768 hybrid key exchange in TLS 1.3 since 2024 [29], [30]. Cloudflare has reported incremental adoption in handshake telemetry [13]. Linux kernel and OpenSSH support is staged across recent releases. Certificate authorities have piloted ML-DSA signing in test hierarchies but have not yet issued production-trust-anchor PQ certificates [31]. The pattern is consistent across operator surveys: the algorithms exist, library support is materializing, but the long tail of embedded, appliance, and legacy software remains unaddressed [10], [11].

2.2 Quantum Key Distribution and ETSI GS QKD 014

Quantum key distribution, in its prepare-and-measure form, was introduced by Bennett and Brassard in 1984 [15] and has been deployed in operational testbeds for two decades [16], [17]. A QKD system distributes symmetric key material between two endpoints using single-photon (or weak-coherent) quantum states, with security guarantees derived from the no-cloning theorem and the disturbance of measurement. Modern systems achieve metropolitan ranges (≤100 km dark fiber) at key rates from kilobits to megabits per second [20], [21]. Trusted-node relaying, satellite QKD [32], and twin-field protocols [33] extend the reach, though at the cost of additional assumptions.

QKD's controversial status in the policy community deserves brief treatment because it informs our observability design choices. The U.S. NSA has stated that QKD is not a replacement for PQC for national-security systems, citing implementation-attack surface, limited authentication coverage, and operational complexity [23]. The French ANSSI takes a similar position [24]. In contrast, the U.K., German, Italian, and Chinese research and standards communities continue to invest in QKD infrastructure, including the EU's EuroQCI program [18] and multiple national QKD networks. The pragmatic operator view, which this paper adopts, is that QKD will be deployed on a subset of high-assurance links (financial inter-datacenter, intelligence, government inter-site) regardless of which policy community is "correct," and that the deployment will be heterogeneous, vendor-specific, and operationally opaque without explicit observability tooling.

ETSI's *GS QKD 014 v1.1.1* specification (2019) standardizes the REST interface between a Key Management Entity (KME) — the device that holds key material produced by the underlying QKD link — and the applications consuming those keys (Secure Application Entities, SAEs) [19]. The interface defines three operations:

- `GET /api/v1/keys/{slave_SAE_ID}/status` returns KME and link status: current key rate, number of stored keys, supported key sizes.
- `GET /api/v1/keys/{slave_SAE_ID}/enc_keys` returns one or more fresh keys from the master KME, identified by UUIDs.
- `POST /api/v1/keys/{master_SAE_ID}/dec_keys` retrieves the matching keys at the slave KME, by UUID.

The specification is deliberately silent on what an SAE does with the keys; in practice, SAEs use them as pre-shared keys (PSK) for IPsec, MACsec, or — increasingly — as the symmetric secret in a hybrid TLS PSK mode [34], [35]. From an observability standpoint, ETSI 014 gives us a stable, vendor-neutral surface on which to observe QKD-protected channels: link status, key rate, request volume, error rates, and (with cooperating SAE instrumentation) which downstream sessions consume the keys.

2.3 Crypto-Agility

Crypto-agility — the property that allows a system to change cryptographic primitives without invasive redesign — was codified operationally by RFC 7696 [25] and is repeatedly named in PQ-migration guidance as a precondition for orderly transition [4], [8], [14]. The literature describes crypto-agility along three dimensions: *protocol* agility (TLS 1.3 negotiates ciphersuites; IPsec IKEv2 negotiates transforms), *code* agility (a library exposes algorithm choice as a parameter, not a recompile-time decision), and *deployment* agility (operators can roll out a new algorithm without firmware replacement or trust-anchor reissuance).

In practice, agility varies dramatically across asset classes. A modern Web server typically negotiates its TLS ciphersuite on every handshake; a hardware HSM may lock its supported key types at the firmware level; embedded devices in industrial or medical settings often have a fixed algorithm chosen at the silicon level. The emergence of FIPS 140-3 validation, with its requirement that algorithm changes trigger revalidation, introduces a second axis of *locked* status that is operationally significant for any regulated environment [36].

Despite repeated standards-body emphasis on crypto-agility, the practitioner picture is poor. No widely deployed inventory tool we have seen reports crypto-agility as a first-class field. The question “can this asset migrate?” is repeatedly answered informally — by tribal knowledge, vendor questionnaires, or case-by-case engineering audits.

---

3. Related Work

We organize related work along three threads — PQC discovery tooling, QKD telemetry, and crypto-agility analysis — and conclude with the gap statement that motivates the present paper.

3.1 PQC Discovery and Migration Tooling

Cisco’s PQC Discovery service [10] is, to our knowledge, the most operationally mature PQ inventory tool deployed today. It combines passive network observation (NetFlow-derived context with optional inline TLS handshake inspection) with active endpoint scanning and a managed-service operator dashboard. The analytic surface is algorithm-class oriented: assets are labeled *PQ-ready*, *classical*, or *unknown* based on observed handshakes and reported algorithm support. The service does not surface channel-protection or agility as separate dimensions.

IBM’s *Quantum-Safe Discover* [11] and Microsoft’s internal *CryptoTracker* tooling [12] follow similar patterns: enumerate cryptographic uses across an environment, classify by algorithm against a PQC-readiness yardstick, prioritize migration. Cloudflare’s published reports [13] focus on edge-observable TLS handshake mixtures and provide one of the few public datasets on PQ adoption on the open Internet.

The academic literature includes systematic measurements of TLS deployment hygiene (key sizes, cipher choices, validation behavior) [37],

[38], [39] and more recent measurements of hybrid PQ deployment in the wild [40]. NIST IR 8547 [4] provides the most authoritative migration playbook and includes a discovery section, but stops short of prescribing telemetry semantics.

Common across this thread: algorithm-class is treated as the primary axis, hybrid/QKD is at most a footnote, and agility is discussed in prose but not surfaced as a measured field.

3.2 QKD Telemetry

QKD telemetry has been studied primarily from the perspective of the QKD operator — the entity running the physical link and the KMEs. The ETSI ISG-QKD has produced multiple documents on operational and security testing [41], [42], [43]; the SECOQC [16] and Tokyo [17] testbeds published detailed operational data. The EuroQCI program [18] is in the process of standardizing inter-domain QKD telemetry.

Crucially, this thread is *internal* to the QKD operator. From the SAE/application perspective — that is, from the perspective of an operator using QKD keys to protect application traffic — the public literature is much thinner. ETSI GS QKD 014 [19] standardizes the SAE-facing interface; we are not aware of an existing open-source SAE-side observability platform that consumes 014 telemetry and integrates it with a broader cryptographic inventory.

3.3 Crypto-Agility Analysis

RFC 7696 [25] is the canonical statement. Subsequent work has analyzed agility properties of individual protocols (TLS, IPsec, S/MIME) and proposed agility frameworks for specific domains (industrial control, embedded), though without operationalising them as a continuous telemetry input.

In the practitioner space, NCC Group has published audit reports on cryptographic-agility deficiencies in deployed products [44]. The OWASP and CIS communities maintain checklists [45], but none of this work, to our knowledge, has been operationalized as a continuous telemetry input feeding a cryptographic-posture dashboard.

3.4 Gap Statement

Each of the three threads is mature in isolation. We are not aware of any published work, open-source tool, or commercial product that:

1. Treats algorithmic resistance, channel protection, and migration agility as three independent telemetry axes,
2. Combines them into a single deadline-adjusted posture metric,
3. Provides a reference implementation that emits all three from a shared event schema,
4. Is reproducible by practitioners without privileged access to commercial scanning services.

This paper closes that gap.

---

4. Threat Model and Operator Assumptions

We assume a *Q-day* adversary: a sufficiently capable fault-tolerant quantum computer capable of executing Shor’s algorithm on RSA-2048-class moduli and elliptic-curve groups of practical size. The adversary’s *capability date* is unknown but treated as a moving deadline; the standards-body consensus places the planning horizon between 2030 and 2040 [3], [4], [8]. Following the harvest-now/decrypt-later threat model [3], we assume that an adversary records classical-encrypted traffic *today* and decrypts it *later* once the CRQC is available. Consequently, traffic encrypted with classical-only KEMs is at risk *now*, not at Q-day.

We assume an operator who:

- Controls or has visibility into the cryptographic surfaces of their environment (TLS termination points, certificate inventory, application source code, host configuration, QKD links if any).
- Operates under a calendar deadline (NIST CNSA 2.0 milestones, regulatory mandates, or contractual obligations) that is fixed, even if individual subdeadlines are uncertain.
- Cannot replace all cryptography at once; migration is staged over months to years.

- Tolerates and benefits from continuous telemetry rather than periodic audits.

We assume an adversary who:

- Records traffic now, decrypts later.
- Cannot break PQ-secure primitives at standard parameter sizes (we accept the NIST standardization process's hardness assumptions).
- Cannot break the no-cloning theorem (we accept the standard QKD security argument); but may exploit implementation-level QKD vulnerabilities — detector blinding, side channels, classical authentication failure — and so QKD protection is observed *probabilistically* by the SAE, not guaranteed.
- May exploit non-cryptographic vulnerabilities; this paper does not address those.

The threat model justifies treating agility as a *risk multiplier*: an asset with low algorithmic resistance but high agility may not be at risk in *real* terms because it can be migrated before the deadline horizon. An asset with low resistance and low agility (e.g., a hardware appliance with hard-coded ECDSA on a five-year refresh cycle) is at *significantly higher* real risk, even if its observed primitive is identical to the agile case.

---

5. The Three-Axis Quantum-Risk Posture Model

We define three independent axes — A, C, G — and a unified deadline-adjusted quantum-risk score $q(asset, t)$.

5.1 Axis A — Algorithmic Resistance

Axis A scores the cryptographic primitives observed on an asset on the scale $[0, 1]$, where 0 corresponds to a primitive that is quantum-trivial (a deprecated or known-broken algorithm) and 1 corresponds to a primitive that is quantum-resistant under the standard NIST hardness assumptions.

Following the existing Sezar V1 rollup (defined in `docs/posture-rollup.md` in the Sezar repository), we classify each primitive into one of five categories:

Category	a value	Examples
`pq`	1.0	ML-KEM-{512,768,1024}, ML-DSA-{44,65,87}, SLH-DSA-*, AES-256-GCM, SHA-256
`pq_hybrid`	0.9	X25519+ML-KEM-768, ECDH-P256+ML-KEM-768
`classical`	0.3	X25519, Ed25519, RSA-2048, ECDSA-P256, AES-128-GCM
`unknown`	0.4	Anything not in the classification table
`deprecated`	0.0	SHA-1, MD5, RSA-1024, RC4, 3DES, DH-1024

For an asset with observed primitives $\{p_i\}$ at roles $\{r_i\}$ with role weights w_{r_i} summing to 1, the axis-A score is

$$A(asset) = \sum_i w_{r_i} \cdot a(p_i)$$

Role weights are calibrated to harvest-now/decrypt-later risk: $w_{\text{sig}} = 0.40, w_{\text{kex}} = 0.30, w_{\text{encrypt}} = 0.20, w_{\text{hash}} = 0.10$ for assets exhibiting all four roles, with re-normalization when only a subset is present.

5.2 Axis C — Channel Protection

Axis C scores the channel through which key material reaches the endpoint, on the scale [0, 1], where 0 corresponds to a channel with no quantum-secure key delivery and 1 corresponds to a channel deriving its session key entirely from QKD-delivered material. Three categorical states cover the realistic deployment options:

State	c value	Description
`classical`	0.0	Session key derived solely from the negotiated KEM.
`qkd_hybrid_psk`	0.7	Session key derived from QKD-PSK XOR negotiated KEM (NIST SP 1800-38A pattern, ETSI 014 SAE).
`qkd_only`	1.0	Session key derived from QKD material alone (rare; MACsec-style transport).

Sub-states allow partial credit when telemetry indicates a QKD link is *degraded* — high QBER, sustained KME unavailability, low key rate — but the SAE has not failed over to classical. We discount the score in proportion to the observed degradation; formal degradation thresholds are deferred to the implementation section (§7.2).

A subtle but important observation: the SAE may *think* it is operating in QKD-hybrid mode while the underlying KME is failing gracefully to a classical fallback. Sezar's role is to observe both layers and surface the discrepancy. We define c on observed ETSI 014 status, not on SAE-reported intent.

5.3 Axis G — Migration Agility

Axis G scores the asset's ability to migrate its primitives on the scale $[0, 1]$, where 0 corresponds to an asset whose primitives can be changed only by physical replacement and 1 corresponds to an asset that negotiates primitives on every session. We define five ordinal levels:

Level	g value	Definition (observable signature)
`negotiated`	1.0	Algorithm selected per-session by protocol negotiation. (TLS 1.3 server, modern SSH server, IKEv2 responder.)
`configurable`	0.75	Algorithm fixed per-deployment but changeable by configuration without code change. (Library config file, environment variable.)
`pinned`	0.50	Algorithm fixed in code; changeable by software upgrade. (Hard-coded algorithm name in application source.)
`locked`	0.20	Algorithm fixed in firmware or by FIPS/compliance binding; changeable only by vendor update or revalidation cycle. (Embedded firmware crypto, FIPS 140-3 validated module under tested-configuration constraint.)
`frozen`	0.0	Algorithm fixed in silicon, ROM, or otherwise unchangeable without hardware replacement. (TPM 1.2 with hard-coded RSA-2048; smart-card hard-wired ECDSA-P256.)

The classification is derived from static analysis of the asset's implementation surface plus, where available, vendor declarations of FIPS validation scope. We detail the scanning methodology in §7.3 and the scoring rubric in §8.3.

5.4 Unified Deadline-Adjusted Quantum Risk

The three axes are independent observables. To collapse them into a single posture metric we adopt the operator's deadline as a fourth input.

Let D be the operator-configured deadline (e.g., 2030-01-01 for NSA CNSA 2.0 browser/server class). Let t be the current date. Define the *deadline tension* $\tau(t) = \max(0, \min(1, 1 - (D - t)/H))$, where H is a horizon constant (we use five years by default). When D is far in the future, $\tau \to 0$ and agility forgives lower algorithmic resistance. As $t \to D$, $\tau \to 1$ and agility no longer compensates because there is insufficient time to migrate.

We define the quantum-risk score as

$$q(asset, t) = 1 - (\alpha \cdot A + \beta \cdot C + \gamma(\tau) \cdot G)$$

with $\alpha + \beta + \gamma(\tau) = 1$. Default weights: $\alpha = 0.5, \beta = 0.2, \gamma(\tau) = 0.3 \cdot (1 - \tau)$, re-normalized when γ shrinks. The agility weight is the only weight that shrinks with deadline tension: an asset with high agility but classical algorithms looks safe today (because γ is large) and looks increasingly unsafe as the deadline approaches (because γ shrinks toward zero). This is the intended behavior: agility is forgiving *now*, not *at the deadline*.

Asset-class weights w_k further weight the asset's contribution to the org-wide posture, with `blockchain_key` weighted higher than `tls_session` (a forged signature against a public-chain key is permanent; a forged session is ephemeral).

5.5 Interpretation and Bounds

The score $q \in [0, 1]$, with 0 = posture is fully aligned with the deadline and 1 = posture is maximally exposed. Operators typically configure alert thresholds at $q > 0.6$ ("must migrate") and $q > 0.3$ ("plan migration").

Two edge cases warrant note. First, a fully PQ asset on a non-QKD-protected channel scores $q = 1 - (0.5 \cdot 1 + 0.2 \cdot 0 + 0.3 \cdot g)$, which is approximately 0.2 for a fully agile asset and 0.5 for a fully frozen asset, both before deadline tension. This is intentional: QKD is a *bonus* on PQ-protected sessions, not a requirement. Second, a fully classical asset on a QKD-protected channel scores approximately 0.35 even with no agility — QKD partially compensates for classical algorithms, reflecting the real-world deployment of QKD on high-assurance links.

6. Event Schema Extensions

We extend the Sezar `crypto_inventory_event v1` schema (defined in `docs/crypto-event-schema.md` in the Sezar repository) with additive, non-breaking fields. Existing consumers that ignore unknown top-level fields continue to function; consumers that opt in to v1.1 gain access to channel-protection and agility observables.

6.1 New Top-Level Fields

```
{
  "schema_version": 1,
  "schema_minor": 1,
  "source_module": "sezar-net",
  "observed_at": "2026-08-15T11:42:03.421Z",
  "asset": { ... },
  "primitives": [ ... ],
  "channel_protection": { ... },  // NEW in 1.1
  "agility": { ... },             // NEW in 1.1
  "posture": { ... }
}
```

`schema_minor` is the first additive use of a minor-version field. Consumers must accept any `schema_minor` ≥ their compiled value and treat unknown top-level fields as opaque.

6.2 `channel_protection`

```
{
  "state": "qkd_hybrid_psk",
  "kme_endpoint": "https://kme-1.dc.example/api/v1",
  "key_id_observed": "9c45e0a2-...",
  "psk_age_seconds": 47,
  "link_qber": 0.018,
  "link_key_rate_bps": 12480,
  "link_health": "ok",
  "degraded_reason": null
}
```

Field	Type	Notes
state	enum	classical / qkd_hybrid_psk / qkd_only.
kme_endpoint	string	ETSI 014 base URL. Omitted when state = classical.
key_id_observed	string	UUID of the consumed key, when reported by the cooperating SAE.
psk_age_seconds	int	Age of the PSK when the session began.
link_qber	float	Quantum bit error rate (0–1).
link_key_rate_bps	int	Average key generation rate over the prior minute.
link_health	enum	ok / degraded / failed.
degraded_reason	string	One-sentence reason when degraded; null otherwise.

The fields are populated by sezar-qkd, which polls the ETSI 014 /status endpoint at configurable intervals. SAE-side fields (key_id_observed, psk_age_seconds) require cooperating instrumentation in the SAE; when absent they are null.

6.3 agility

```
{
  "level": "configurable",
  "level_score": 0.75,
  "evidence": [
    {
      "type": "config_pattern",
      "file": "/etc/nginx/nginx.conf",
      "line": 142,
      "snippet": "ssl_protocols TLSv1.2 TLSv1.3;
             \nssl_ciphers HIGH:..."
    },
    {
      "type": "fips_mode",
      "detected": false
    }
  ],
  "scanner_version": "sezar-agility/0.3.1",
  "rubric_version": "qra-rubric/v1.0"
}
```

Field	Type	Notes
level	enum	negotiated / configurable / pinned / locked / frozen.
level_score	float	Numeric value per §5.3.
evidence	array	One or more evidentiary findings supporting the level.
scanner_version	string	Sezar-agility version that produced the score.
rubric_version	string	Version of the public scoring rubric (§8.3).

Evidence types in V1: protocol_negotiation (observed wire-level algorithm negotiation), config_pattern (configuration file exposing algorithm choice), code_pattern (source code reference to a fixed algorithm), firmware_string (binary-extracted algorithm name), fips_mode (FIPS provider/kernel mode detected), vendor_declaration (operator-provided vendor statement).

6.4 New Asset Kinds

```
qkd_link        // identity = KME endpoint URL hash
qkd_kme         // identity = KME ID per ETSI 014 status
```

These are emitted by sezar-qkd independently of the session events that consume their keys. They allow the dashboard to render a QKD link health view distinct from the SAE-side session view.

6.5 Backwards Compatibility

The schema extension is strictly additive:

- All fields new in v1.1 are top-level; no existing field shape changes.
- v1.0 consumers that do not recognize the new fields ignore them.
- v1.1 producers that lack data for the new fields emit them as `null` (per the V1 module emission contract).
- The posture engine treats `channel_protection: null` as `state: classical` and `agility: null` as `level: unknown` (which maps to `level_score: 0.4`, mid-low, the same convention Axis A uses for unknown primitives).

---

7. Sezar: Reference Architecture and Implementation

Sezar is an open-source observability platform implementing the three-axis posture model. The implementation is a single Rust workspace containing seven crates: five agents emitting events, one shared rollup library, and one collector/server.

7.1 Workspace Layout

```
sezar/
├── crates/
│   ├── sezar-core/      # event schema, rollup engine (no I/O)
│   ├── sezar-server/    # axum collector + REST API
│   ├── sezar-net/       # eBPF agent: TLS, SSH, IPsec
│   ├── sezar-qkd/       # ETSI GS QKD 014 collector + emulator
│   ├── sezar-cert/      # X.509 inventory (CT, host scan)
│   ├── sezar-chain/     # public-chain crypto observation
│   ├── sezar-id/        # HSM/KMS/smart-card inventory
│   └── sezar-agility/   # static crypto-agility scanner
├── docs/
└── web/                 # React + Vite dashboard
```

The architectural invariant — every agent emits one and only one event shape, computed locally via `sezar-core::rollup` — is what keeps the platform composable across surfaces as different as eBPF TLS sniffing and Solidity source code analysis.

7.2 sezar-qkd: ETSI 014 Collector and Emulator

The `sezar-qkd` crate fulfils two roles. Operationally, it is a collector that polls one or more ETSI GS QKD 014 KMEs, emits `qkd_link` and `qkd_kme` events, and serves as the data source for the `channel_protection` block on session events emitted by cooperating SAEs.

7.2.1 Collector design

The collector is a Tokio async loop that, for each configured KME, issues:

- `GET /api/v1/keys/{slave_SAE_ID}/status` at a configurable cadence (default 5 s).
- An auxiliary `enc_keys` request at a slower cadence (default 60
   s. to measure end-to-end key delivery latency. Keys are requested with `size=0` when permitted, or discarded when not.

The collector tracks per-KME state (last good status time, exponentially-weighted error rate, QBER history) and emits `qkd_link` events on status change, plus heartbeat events at a configurable interval. Authentication to the KME follows ETSI 014 guidance: mutual TLS with the SAE certificate.

7.2.2 Emulator

Hardware QKD is expensive and rare; reproducibility requires an alternative. We implement `sezar-qkd-kme-emulator`, a faithful implementation of ETSI 014 v1.1.1 backed by a synthetic key generator. The emulator:

- Implements `/status`, `/enc_keys`, and `/dec_keys` exactly per spec.
- Generates synthetic keys at a configured rate, with configurable QBER, key size, and lifetime.

- Supports replay scenarios — pre-recorded sequences of link state changes (degradation, failure, recovery) — for reproducibility.
- Logs every interaction in a documented JSON capture format enabling head-to-head A/B testing of SAE implementations.

The emulator is the foundation of the §8.2 empirical study and is released alongside Sezar as a standalone tool.

7.3 sezar-agility: Static Crypto-Agility Scanner

The `sezar-agility` crate implements the agility-axis scoring per §5.3. It accepts as input one or more *targets* and produces `agility` blocks attached to the corresponding assets.

7.3.1 Target types

Target	Evidence sources
Source repository	Semgrep ruleset over the language-specific config and source files; file-path heuristics for build-time pinning.
Installed package	Package manifest (rpm, dpkg, pip, npm) + binary string-extraction over the installed artifacts.
Running host	TLS handshake observation against the host (server algorithm support → `negotiated` evidence); plus optional auth into the host's config files.
Vendor appliance	Vendor-declared algorithm scope + observed handshake behavior.

7.3.2 Ruleset

The published ruleset (`sezar-agility/rules/v1`) consists of several hundred Semgrep patterns covering common cryptographic libraries and protocols across C, Go, Rust, Python, Java, and configuration formats (nginx, Apache, OpenSSL config, sshd_config, strongswan.conf, Postfix, Dovecot, HAProxy, Envoy). Each pattern emits one piece of evidence with a documented mapping to the five-level rubric.

7.3.3 Scoring algorithm

For each target, the scanner collects evidence and applies a **most-agile-wins** aggregation: the asset is scored at the *most* agile level supported by any rule that fired. The rationale is asymmetric: rules fall into two semantic classes. *Capability* rules (e.g., presence of an `ssl_ciphers` directive, a `SSL_CTX_set_cipher_list` call) demonstrate that an operator can in fact change the algorithm without invasive surgery; their emit_level is an *upper bound on what is observable*. *Constraint* rules (e.g., a literal hard-coded algorithm name in source) only report that *some* algorithm is fixed somewhere — they do not imply that there is no overriding config surface. Conservative-min aggregation systematically misclassifies large agility projects like nginx (which simultaneously embeds default algorithm strings in source and exposes ten configuration knobs that override them); the operationally useful question is “what is the best surface I have,” not “what is the worst constraint anywhere.”

Conservative-min remains available as an alternative aggregation projection for operators who require it (regulatory contexts where the weakest surface governs); the published scanner exposes both projections through a dashboard toggle.

Where evidence is absent, the scanner produces `level: pinned` (the documented `UNKNOWN_LEVEL_FALLBACK`) and surfaces the issue for operator review.

We discuss limitations of static-only agility scoring in §9.

7.4 sezar-core Unified Rollup

The rollup engine extends the V1 implementation (see `docs/posture-rollup.md` in the Sezar repository) with the deadline-adjusted three-axis formula of §5.4. The implementation remains a pure function with no I/O; the operator configures D, H, and asset-class weights via the dashboard, and the engine recomputes on every event ingest. Fuzz testing covers all axis combinations.

7.5 sezar-server and Dashboard

The collector is an Axum HTTP service accepting v1.0 and v1.1 events, validating against a generated JSON schema, persisting to Postgres (configuration) and a columnar store (events), and serving a React+Vite dashboard. The dashboard renders a three-axis posture matrix per asset class, a deadline-countdown view tied to the configured D, and an inventory table sortable by q. Implementation details are routine and we defer them to the project documentation.

---

8. Empirical Evaluation

We evaluate the model and implementation through three studies, each designed to be reproducible by an external practitioner with the published rulesets, emulator, and corpus lists.

8.1 Study 1 — Axis A on the Public Web (Tranco-top-1k)

8.1.1 Methodology

We scan the Tranco-top-1k [46] over TLS using a published scanner script that wraps `zgrab2` [47] in two modes: a *baseline* handshake (no PQ groups offered) and a *PQ-capable* handshake (offering X25519MLKEM768 alongside classical groups). For each host we record: negotiated cipher suite, negotiated key share, server-supported groups, certificate signature algorithm, certificate validity. The scan is deliberately constrained to one TCP connection per host, identifies itself in the User-Agent and ClientHello SNI extension as `sezar-survey/1.0 +https://e2esolutions.tech/sezar`, and respects robots.txt for the SNI hostname where applicable.

The scan source code, target list, raw `zgrab2` outputs, and analysis notebooks will be released alongside this paper.

Ethical considerations: this is a one-shot benign TLS handshake similar to common research scans [37]; we do not attempt protocol downgrade or repeated probing. Scan rate is ≤10 Hz and exits immediately on connection error.

8.1.2 Metrics

We report the distribution of asset-A scores across the corpus, the prevalence of PQ-capable hosts, the distribution of certificate signature algorithms, and the prevalence of weak/deprecated primitives still observed.

8.1.3 Results (n = 30 sample)

A 30-host pilot of the methodology — covering major content-delivery, browser-vendor, distro, IETF/IEEE/NIST/ETSI, and AI-vendor properties — produced the following baseline. Two probes ran sequentially over the same host list (full sources and raw captures at `studies/study1/`):

1. **Classical baseline probe** — Python `ssl` with the system OpenSSL defaults. Establishes a TLS 1.3 handshake without advertising `X25519MLKEM768`.
2. **PQ-capable probe** — Rust binary built on `rustls = 0.23` + `rustls-post-quantum = 0.2`, with a custom certificate verifier that accepts every chain (observability, not authentication). Advertises `X25519MLKEM768` alongside the classical groups, captures the negotiated kex group, ciphersuite, and leaf cert signature algorithm.

Observable	Classical probe	PQ-capable probe
TLS 1.3 negotiation	30/30 (100%)	30/30 (100%)
AES-256-GCM/SHA-384	20/30 (67%)	20/30 (67%)
AES-128-GCM/SHA-256	7/30 (23%)	9/30 (30%)
ChaCha20-Poly1305/ SHA-256	3/30 (10%)	1/30 (3%)
ECDSA leaf cert	18/30 (60%)	16/30 (53%)
RSA-PKCS1 leaf cert	12/30 (40%)	14/30 (47%)
ML-DSA / SLH-DSA cert	0/30	0/30
Deprecated primitive (SHA-1, RC4)	0/30	0/30
`X25519MLKEM768` negotiated	n/a	**17/30 (57%)**

Small ciphersuite / cert-sig diffs between the two probes reflect SNI-driven host-selection variance (a given site may front different certs from different POPs) rather than substantive disagreement.

The headline PQ-adoption result is 17/30 (57%). The PQ-adopter cohort includes Cloudflare-fronted sites (cloudflare.com, twitter.com, reddit.com, anthropic.com, openai.com, e2esolutions.tech), Google properties (google.com, youtube.com), and a long tail of community / CDN-protected sites (wikipedia.org, facebook.com, instagram.com, apple.com, python.org, rust-lang.org, debian.org, ietf.org, etsi.org). The 13 classical-only hosts include several major infrastructure-relevant properties — github.com, microsoft.com, amazon.com, mozilla.org, kernel.org, nist.gov, openssl.org — where the PQ rollout has not yet landed at the date of measurement (2026-05-13).

The 57% in this curated sample is meaningfully higher than open-web averages reported by Cloudflare in 2024–2025 (≈25–30% across observed handshakes at the edge) [13], because the sample skews toward Cloudflare-fronted and Google-fronted properties whose edge terminators rolled out X25519MLKEM768 early. The sample size is too small to draw distributional conclusions about the wider Internet; the contribution here is methodological — a reusable, ethics-vetted PQ-aware probe (`sezar-net pq-probe`) that emits NDJSON suitable for direct ingestion into the Sezar collector. Scaling to the Tranco-top-1k is mechanical; the rate-cap and ethical safeguards stay intact.

Pipeline integration: PQ-probe NDJSON parses straight into the same `tls_session` asset shape consumed by `from-zgrab`, so the same downstream rollup, BLOCKED-flag derivation, and dashboard render exactly as for any other source-module input.

8.2 Study 2 — Axis C via the ETSI GS QKD 014 Emulator

8.2.1 Methodology

We construct a controlled testbed of one master KME, two slave KMEs, and three cooperating SAEs (a strongSwan IPsec endpoint operating in PSK mode with QKD-PSK rotation, a Wireguard endpoint with manual PSK rotation, and a custom TLS endpoint using the NIST SP 1800-38A hybrid-PSK pattern). The KMEs are `sezar-qkd-kme-emulator` instances; we drive them through a documented sequence of replay scenarios:

- **R1 — Steady-state.** Constant QBER 1.8%, key rate 12 kbps, no interruption. 24 hours.
- **R2 — Gradual degradation.** QBER ramps from 1.8% to 8.5% over 4 hours; the SAE policy should fail over to classical at the configured threshold. We observe whether each SAE detects the degradation and whether Sezar's `link_health` reflects the transition.
- **R3 — Hard failure.** KME unreachable for 30 minutes. SAE behavior should be: continue with cached PSK until lifetime expires, then fail closed or fall back to classical per policy.
- **R4 — Stale PSK.** PSK rotation is suppressed at the SAE while the KME continues to produce keys. Sezar should observe `psk_age_seconds` rising past policy.

- **R5 — Bifurcated SAE.** The strongSwan endpoint sees a healthy KME while the Wireguard endpoint sees a failed KME (simulating partial KME outage). Sezar should report inconsistent per-session `channel_protection` while the `qkd_link` aggregate is `degraded`.

8.2.2 Metrics

We measure: SAE failover correctness, Sezar observation latency (emulator change → emitted event), event-ordering correctness under concurrent KME polls, and posture-rollup correctness (particularly that the unified q score reflects the degradation appropriately).

8.2.3 Results

All five scenarios ran end-to-end in compressed time (30–60s each rather than the full 4–24h documented duration; the runner is at `studies/study2/run.sh`). Per-scenario captures and the analysis script are at `studies/study2/`.

Classification correctness: 13/13. Every operator-induced state change produced a downstream `link_health` reading matching the expected post-op state.

Scenario	Events captured	Induced transitions	Matched
R1 — steady-state	33	1	1/1
R2 — ramp	63	5	5/5
R3 — hard-failure	48	3	3/3
R4 — stale-PSK	33	1	1/1
R5 — bifurcated	48	3	3/3

Observation latency: p50 = 0.71s, range 0.70–0.71s across all 13 transitions, against a configured 1-second poll interval. The latency band hovers tightly around half the poll period — the analytical expectation for periodic polling — and dominates over event-emission cost. Increasing the poll interval moves the median latency linearly; in real ETSI 014 deployments where the operational poll cadence is typically 5–10 s [41], this gives a p50 closer to 2.5–5 s.

R3's link_health timeline is shown in Figure 6 of the magazine version of this paper; the analogue plot for every scenario ships in `studies/study2/plots/`. R5 confirms the per-session attribution thesis: when one KME returns 503 the link-level event flips to `failed` even though a separately healthy paired KME continues to deliver keys — an operator running KME-only telemetry cannot distinguish this case from a full outage, and the channel-protection block on per-session events is the diagnostic surface that resolves it.

R4 reveals a deliberate observability gap. Sezar's KME-side polling alone cannot distinguish a fresh PSK from one the SAE has held for hours; the `psk_age_seconds` field on `channel_protection` is populated only when the SAE co-operates. Closed-source SAEs without that instrumentation are visible to Sezar only at the link layer. We open-source patches for strongSwan, Wireguard, and a sample TLS endpoint to populate the field.

The emulator, replay scripts, and analysis notebooks ship under MIT alongside Sezar.

8.3 Study 3 — Axis G on Fifty Open-Source Server Projects

8.3.1 Methodology

We select fifty widely deployed open-source server projects spanning HTTP, mail, database, message broker, and VPN categories: nginx, Apache httpd, HAProxy, Envoy, Caddy, Traefik, OpenSSH server, Postfix, Dovecot, Exim, PostgreSQL, MySQL, Redis, MongoDB, RabbitMQ, Kafka, OpenVPN, strongSwan, Wireguard, PowerDNS, BIND, Unbound, CoreDNS, and others. The full list, with hand-graded ground-truth levels and reviewer notes, ships as `crates/sezar-agility/corpus/oss-50-v1.csv` in the Sezar repository.

For each project we:

1. Run `sezar-agility` against the source repository at a pinned commit.

2. Run `sezar-agility` against the installed binary on Rocky Linux 10 with default package configuration.
3. Hand-grade the project against the §5.3 rubric, using two reviewers and reporting inter-rater agreement.
4. Report the divergence between automatic and hand-graded scores; treat the hand grade as ground truth for the corpus.

8.3.2 Metrics

We report per-project agility level, the per-evidence contribution to the score, the false-negative and false-positive rates of the Semgrep ruleset against the ground-truth grading, and the distribution of agility levels across the corpus by category.

8.3.3 Results (n = 11 pilot subset)

The full OSS-50 corpus is committed at `crates/sezar-agility/corpus/oss-50-v1.csv` with hand-graded ground truth. A pilot run over an 11-project subset spanning nine categories (HTTP, mail, DB, message-broker, DNS, VPN/secure-shell, messaging, certificate-authority, time) exercises the full pipeline:

Project	Category	Hand-grade	Scanner	Match
nginx	http_server	configurable	configurable	yes
haproxy	http_server	configurable	configurable	yes
caddy	http_server	configurable	configurable	yes
unbound	dns_server	configurable	configurable	yes
coredns	dns_server	configurable	configurable	yes
postfix	mail_server	configurable	configurable	yes
redis	database	configurable	configurable	yes
nats-server	message_broker	configurable	configurable	yes
step-ca	certificate_authority	configurable	configurable	yes
prosody	messaging	configurable	configurable	yes
wireguard-tools	vpn_secure_shell	pinned	pinned	yes
chrony	time	configurable	**pinned**	no

Agreement: 10/11 (91%); Cohen's κ = 0.62 (substantial agreement on the Landis–Koch scale). The single dissent is **chrony**, where the static evidence is genuinely ambiguous between a `configurable` reading (NTS key-types are operator-controlled in `chrony.conf`) and a `pinned` reading (the NTP authentication path embeds many literal symmetric- algorithm references). The scanner picked the latter on the strength of 11 hard-coded references; no capability rule fired on chrony's NTS surface because it uses neither OpenSSL nor Go's `crypto/tls`. We flag this for v2 of the ruleset and the corpus.

The confusion matrix in Figure 7 of the magazine version visualises this result. Per-project event JSON, evidence listings, and the full agreement TSV are at `studies/study3/results/`.

Failure modes characterised during the run:

- **Go ecosystem coverage gap.** The v1 ruleset's first cut matched only OpenSSL-style API calls (`SSL_CTX_set_cipher_list`, `SSL_CONF_cmd`, …). Three projects (coredns, step-ca, caddy) initially classified as `pinned` because nothing matched. Adding a one-rule `go-stdlib-tls-config` pattern matching `crypto/tls`'s `Config` / `CipherSuites` / `MinVersion` fields recovered all three to `configurable`.
- **Aggregation policy.** Conservative-min aggregation systematically misclassified nginx: nginx embeds literal algorithm names in source (for crypto-impl code paths and tests, emit_level `pinned`) and exposes operator config knobs (emit_level `configurable`). The §7.3.3 most-agile- wins policy resolves the ambiguity in favour of the capability surface; conservative-min remains available as a regulatory-context alternative.
- **No-evidence fallback.** Wireguard correctly classified `pinned` with zero evidence collected — the project's userspace tooling has no OpenSSL or Go-tls surface, so the documented `UNKNOWN_LEVEL_FALLBACK = Pinned` applies. The ground-truth grade agrees.

Scaling the pilot to the full OSS-50 list is mechanical; the runner already iterates over the corpus CSV.

8.4 Synthesis: End-to-End Pipeline

`scripts/demo.sh` exercises the full V1 pipeline end-to-end: boot the KME emulator, the QKD collector, sezar-server, and seed events from both the bundled zgrab2 fixture and a synthetic FIPS-locked asset. The collector's `/v1/posture` endpoint returns:


```json
{
 "org_q": 0.627,
 "deadline": "2030-01-01T00:00:00Z",
 "horizon_years": 5.0,
 "assets": 5,
 "blocked_count": 1
}
```


The inventory ordering, sorted by *q* descending, places the TLS 1.0 + RC4 + SHA-1 legacy host at the top (q ≈ 0.72), the TLS 1.2 ECDHE+RSA classical host just below (q ≈ 0.69), the FIPS-locked appliance third with the `BLOCKED` flag raised (q ≈ 0.68), the modern TLS 1.3 + ECDSA-P256 hybrid-PSK QKD host at the bottom of the priority queue (q ≈ 0.43). This matches the model's expected behaviour in §3.1's worked example: the channel-protection axis pulls the QKD-protected asset down even though its algorithmic content is classical, the agility axis pushes the FIPS-locked asset up even though its observed primitives match the nominally-agile modern host, and the deadline-tension term ensures the legacy host dominates the queue.

The implementation's pure-Rust rollup (in `crates/sezar-server/src/posture.rs`) was verified against the §3.1 worked example: the unit tests `worked_example_alpha_q_matches_paper` and `worked_example_delta_q_matches_paper` assert that the implementation reproduces the paper's α = 0.544 and δ = 0.392 to within 0.01.

This is the first published end-to-end empirical pipeline on all three axes whose every leg — scan, collect, rollup, dashboard — ships as open source and runs on a single Linux host with no QKD hardware and no commercial scanner. Scaling to the full Tranco-1k and the OSS-50 corpus is a mechanical exercise on the published runners; the small-sample numbers above demonstrate the methodology, not the final headline.

---

9. Discussion

9.1 Limitations

Our agility scoring is static and pattern-based; it is necessarily approximate. A project may *appear* pinned in source while actually being agile through a runtime extension mechanism we did not pattern-match. Conversely, a project may appear agile via a config field that is in practice never changed. We address false negatives by reporting per-evidence detail; we cannot fully address the second class without operator input.

Our channel-protection axis depends on cooperating SAE instrumentation for per-session attribution. Sezar can observe the KME state independently, but linking a specific TLS or IPsec session to a specific consumed key requires the SAE to emit the `key_id_observed` field. We expect adoption in cooperating software (we open-source patches for strongSwan, Wireguard, and a sample TLS endpoint as part of the release) but note that closed-source SAEs may report only at the link level.

Our threat model accepts the NIST PQC hardness assumptions and the standard QKD security argument. Compromise of either — by mathematical break (PQC) or by implementation attack (QKD) — would require recalibration of the scoring tables. The schema supports this: the rollup constants are operator-tunable in configuration, not compiled in.

We do not address economic or political constraints on migration (budget cycles, regulatory approval lag, vendor support windows). These are first-order operator concerns and we acknowledge them as out of scope for the observability layer; they are downstream consumers of the posture data.

9.2 Ethical Considerations

Active TLS scanning, even of public web hosts, must be conducted responsibly. We follow established practice from prior measurement work [37]: one connection per host, clear User-Agent identification, opt-out instructions linked from the identifying URL, conservative scan rate, no protocol downgrade, no repeated probing. The Tranco list excludes adult content and known sensitive categories; we additionally exclude any host that returns a robots.txt directive on its base URL within the first 1024 bytes.

The agility scanner operates on source code already public under open-source licenses and on installed binaries on hosts the operator controls. No private code or vendor materials are processed.

The QKD emulator generates synthetic key material only; it is not connected to a real QKD link in our published study. Operators integrating real KMEs are responsible for the security of those KMEs and the surrounding network, including mutual TLS authentication of the SAE.

9.3 Deployment Guidance

For operators planning to deploy Sezar against a real environment, we recommend a phased rollout:

1. **Phase 1: Inventory only.** Run `sezar-net` and `sezar-agility` in observe-only mode. Establish a baseline.
2. **Phase 2: Add deadline.** Configure D per applicable regulatory regime. Observe how q evolves with no other change.
3. **Phase 3: Prioritize.** Sort assets by q descending, migrate the top N by quarter.
4. **Phase 4: Integrate QKD telemetry where it exists.** Add `sezar-qkd` only when a real KME is present and the SAE instrumentation is in place; the channel-protection axis is *additive* and never required.

We caution against treating q as a primary KPI. It is a *comparative* score, calibrated for relative prioritization within an environment. Inter-organization comparison requires shared D, shared weights, and shared corpora.

9.4 Future Work

Several extensions are natural and intentionally deferred:

- **Time-decay.** Treating an observation made three months ago as less authoritative than one made today.
- **Asset relationship modeling.** Linking a TLS session's certificate to the issuing CA's signing key allows the posture of an upstream asset to propagate into the downstream score.
- **Adversary modeling.** Allowing operators to plug in their own estimate of CRQC arrival as a probability distribution over D rather than a single date.
- **Standardization.** Submitting the `crypto_inventory_event` schema to IETF as an Informational draft so other tools can emit and consume it.

---

10. Conclusion

Quantum-risk posture is not a one-axis question. The widely deployed framing — "is my asset PQ-ready?" — answers a question strictly easier than the one operators actually need to answer: "will my asset be PQ-ready *in time*, given everything I know about its algorithms, its channel, and its migration cost?" We have argued for a three-axis posture model that surfaces algorithmic resistance, channel protection, and migration agility as independent observables, and combines them with the operator's deadline into a single quantum-risk score. We have specified an extension to an open event schema, presented a reference implementation across five cooperating agents, and designed three empirical studies — each fully reproducible by practitioners — that establish a first baseline against real-world corpora. The schema, the implementation, and the empirical artifacts (TLS scan harness, ETSI 014 emulator, agility ruleset and hand-graded corpus) are released under MIT alongside this paper.

We do not claim that Sezar's scoring constants are optimal, that the agility rubric covers every deployment pattern, or that our threat model survives every adversary. We do claim that surfacing all three axes is a strict improvement over the current state of practice, and that the released artifacts allow the community to critique, refine, and supersede our specific choices on a shared empirical footing.

Acknowledgments

[To be added.]

References

[1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997, doi: 10.1137/S0097539795293172.

[2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the 28th annual ACM symposium on theory of computing (STOC)*, 1996, pp. 212–219. doi: 10.1145/237814.237866.

[3] NSA, “Announcing the Commercial National Security Algorithm Suite 2.0,” U.S. National Security Agency, Sep. 2022.

[4] NIST, “Transition to Post-Quantum Cryptography Standards,” National Institute of Standards; Technology, NIST IR 8547 (Initial Public Draft), Nov. 2024.

[5] NIST, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM),” National Institute of Standards; Technology, Aug. 2024.

[6] NIST, “FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA),” National Institute of Standards; Technology, Aug. 2024.

[7] NIST, “FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA),” National Institute of Standards; Technology, Aug. 2024.

[8] UK NCSC, “Preparing for Quantum-Safe Cryptography.” UK National Cyber Security Centre guidance, 2023.

[9] ANSSI, “ANSSI Views on the Post-Quantum Cryptography Transition,” Agence nationale de la sécurité des systèmes d’information (France), 2022.

[10] Cisco Systems, “Post-Quantum Cryptography at Cisco: Trust Center Overview and Quantum-Safe WAN Whitepaper.” https://www.cisco.com/site/us/en/about/trust-center/post-quantum-cryptography.html, 2024.

[11] IBM, “IBM Guardium Quantum Safe and IBM Quantum Safe Explorer.” https://www.ibm.com/products/guardium-quantum-safe, 2024.

[12] A. Shemesh and J. Rutzer, “Building your cryptographic inventory: A customer strategy for cryptographic posture management.” Microsoft Security Blog, 2026-04-16, https://www.microsoft.com/en-us/security/blog/2026/04/16/building-your-cryptographic-inventory-a-customer-strategy-for-cryptographic-posture-management/, Apr. 2026.

[13] B. Westerbaan, “The state of the post-quantum Internet.” Cloudflare Blog, 2024-03-05, https://blog.cloudflare.com/pq-2024/, Mar. 2024.

[14] BSI, “Migration to Post-Quantum Cryptography: Recommendations for Action,” Bundesamt für Sicherheit in der Informationstechnik (Germany), 2023.

[15] C. H. Bennett and G. Brassard, “Quantum cryptography: Public key distribution and coin tossing,” in *Proceedings of the IEEE international conference on computers, systems and signal processing*, Bangalore, India, 1984, pp. 175–179.

[16] M. Peev, C. Pacher, R. Alléaume, C. Barreiro, J. Bouda, *et al.*, “The SECOQC quantum key distribution network in vienna,” *New Journal of Physics*, vol. 11, no. 7, p. 075001, 2009.

[17] M. Sasaki, M. Fujiwara, H. Ishizuka, *et al.*, “Field test of quantum key distribution in the Tokyo QKD Network,” *Optics Express*, vol. 19, no. 11, pp. 10387–10409, 2011.

[18] European Commission, “European Quantum Communication Infrastructure (EuroQCI) Initiative.” https://digital-strategy.ec.europa.eu/en/policies/european-quantum-communication-infrastructure-euroqci, 2023.

[19] ETSI ISG-QKD, “GS QKD 014 V1.1.1 — Quantum Key Distribution; Protocol and Data Format of REST-Based Key Delivery API,” ETSI, Feb. 2019.

[20] N. Aquina *et al.*, “A Critical Analysis of Deployed Use Cases for Quantum Key Distribution and Comparison with Post-Quantum Cryptography.” IACR Cryptology ePrint Archive 2025/173, https://eprint.iacr.org/2025/173, 2025.

[21] M. Pittaluga *et al.*, “600-km repeater-like quantum communications with dual-band stabilization,” *Nature Photonics*, vol. 15, pp. 530–535, 2021, doi: 10.1038/s41566-021-00811-0.

[22] Quantum Telecommunications Italy (QTI), "QUID: Italian National Quantum Communication Infrastructure Deployment." EuroQCI Italy project, https://quid-euroqci-italy.eu/the-project/, 2024.

[23] NSA, "Quantum Key Distribution (QKD) and Quantum Cryptography (QC)." NSA Information Sheet, 2020.

[24] ANSSI, "Should Quantum Key Distribution Be Used for Secure Communications?" Agence nationale de la sécurité des systèmes d'information (France), 2020.

[25] R. Housley, "Guidelines for Cryptographic Algorithm Agility and Selecting Mandatory-to-Implement Algorithms," IETF, RFC 7696, 2015.

[26] NIST, "Post-Quantum Cryptography Standardization Call for Proposals." https://csrc.nist.gov/projects/post-quantum-cryptography, 2016.

[27] R. Perlner, "FIPS 206 Status Update: FN-DSA (Falcon)," National Institute of Standards; Technology, Computer Security Division, 2025.

[28] NIST, "Additional Digital Signature Schemes and KEM On-Ramp Round." NIST Post-Quantum Cryptography project page, 2023.

[29] D. Adrian, B. Beck, D. Benjamin, and D. O'Brien, "Advancing Our Amazing Bet on Asymmetric Cryptography." Chromium Blog, 2024-05-23 (Chrome 131 shipped ML-KEM Nov 2024), https://blog.google/chromium/advancing-our-amazing-bet-on-asymmetric/, May 2024.

[30] Mozilla, "Firefox 132 Release Notes: Post-Quantum Key Exchange (mlkem768x25519) for TLS 1.3." https://www.firefox.com/en-US/firefox/132.0/releasenotes/, Oct. 2024.

[31] DigiCert, "ML-DSA and FN-DSA Post-Quantum Signature Readiness." DigiCert PQC Insights and Blog, https://www.digicert.com/insights/post-quantum-cryptography/mldsa, 2025.

[32] S.-K. Liao, W.-Q. Cai, W.-Y. Liu, *et al.*, "Satellite-to-ground quantum key distribution," *Nature*, vol. 549, pp. 43–47, 2017.

[33] M. Lucamarini, Z. L. Yuan, J. F. Dynes, and A. J. Shields, "Overcoming the rate–distance limit of quantum key distribution without quantum repeaters," *Nature*, vol. 557, pp. 400–403, 2018.

[34] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, and N. Sullivan, "KEM-based pre-shared-key handshakes for TLS 1.3." IETF Internet-Draft draft-wiggers-tls-authkem-psk-05 (2026-05-04, individual submission), https://datatracker.ietf.org/doc/draft-wiggers-tls-authkem-psk/, May 2026.

[35] NIST National Cybersecurity Center of Excellence, "Migration to Post-Quantum Cryptography (NIST SP 1800-38, Vols. A–C, Preliminary Drafts)," National Institute of Standards; Technology, 2023.

[36] NIST, "FIPS 140-3: Security Requirements for Cryptographic Modules," National Institute of Standards; Technology, 2019.

[37] Z. Durumeric, J. Kasten, M. Bailey, and J. A. Halderman, "Analysis of the HTTPS certificate ecosystem," in *Proceedings of the ACM internet measurement conference (IMC)*, 2013.

[38] R. Holz, L. Braun, N. Kammenhuber, and G. Carle, "The SSL landscape: A thorough analysis of the X.509 PKI using active and passive measurements," in *Proceedings of the ACM internet measurement conference (IMC)*, 2011.

[39] A. P. Felt, R. Barnes, A. King, C. Palmer, C. Bentzel, and P. Tabriz, "Measuring HTTPS adoption on the web," in *Proceedings of the 26th USENIX security symposium*, 2017.

[40] Cloudflare Research, "State of the Post-Quantum Internet in 2025." Cloudflare Blog, https://blog.cloudflare.com/pq-2025/, 2025.

[41] ETSI ISG-QKD, "GS QKD 002 — Use Cases," ETSI, 2010.

[42] ETSI ISG-QKD, "GS QKD 008 — Module Security Specification," ETSI, 2010.

[43] ETSI ISG-QKD, "GS QKD 011 — Component Characterization: Characterizing Optical Components for QKD Systems," ETSI, 2016.

[44] NCC Group, "Cryptographic Implementation Audits and Crypto-Agility Findings (Public Reports)." NCC Group Research blog, https://research.nccgroup.com/, 2018--2024.

[45] OWASP Foundation, "OWASP Cryptographic Storage Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html, 2024.

[46] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, “Tranco: A research-oriented top sites ranking hardened against manipulation,” in *Proceedings of the network and distributed system security symposium (NDSS)*, 2019.

[47] ZMap Project, “zgrab2: A Fast Application-Layer Network Scanner.” https://github.com/zmap/zgrab2, 2024.